PYTHON PROJECT REPORT

HackerRank Python Problem Solving

Complete Practice & Solution Portfolio
Based on the supplied Data Python Queries roadmap

HackerRank Badge: - Raja Kumar - rajakumarintech | HackerRank

Prepared by: Raja Kumar

Platform: HackerRank

Supporting Resources: Google Colab and Kaggle

1. Project Overview

This project is a structured Python programming practice portfolio based on the supplied Data Python Queries roadmap. The worksheet lists beginner and medium-level HackerRank exercises covering Python fundamentals, data types, operators, conditional statements, loops, functions, collections, and string manipulation. The aim is to practise each concept through a problem-solving approach and build a foundation for further Python use in data analytics and automation.

The supplied roadmap identifies Google Colab and Kaggle as learning resources and describes a five-star HackerRank rating as the practice goal. This report documents the problem set and provides solution patterns; it does not claim that a particular rating has already been achieved.

2. Project Objectives

- Learn Python syntax and core data types.
- Practise variables, arithmetic, conditions, and loops.
- Understand functions, lists, tuples, dictionaries, and mappings.
- Develop string manipulation and formatting skills.
- Solve beginner and medium-level programming problems.
- Build reusable problem-solving patterns for coding practice.

3. Tools and Resources

- Python — programming language is used for the solutions.
- HackerRank — coding practice and test-case validation platform.
- Google Colab — browser-based notebook for writing and running Python.
- Kaggle — additional programming and data practice resource.

4. Python Problem Solutions

Each solution below is written to match the common HackerRank problem format associated with the question title. Use one solution at a time in the corresponding challenge. If the HackerRank editor provides starter code, keep its required function signature and avoid duplicating input handling.

5.1 Day 0: Hello, World

Objective: Print the required greeting and the input string.

```
print ("Hello, World.")
s = input()
print(s)
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.2 Day 1: Data Types

Objective: Read the provided values, then demonstrate integer addition, float addition, and string concatenation.

```
i = 4
d = 4.0
s = "HackerRank "

i2 = int (input ())
d2 = float (input ())
s2 = input ()

print (i + i2)
print (d + d2)
print (s + s2)
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.3 Day 2: Operators

Objective: Calculate the total meal cost including tips and tax, rounded to the nearest integer.

```
meal_cost = float (input (). strip ()
tip_percent = int (input (). strip ()
tax_percent = int (input (). strip ())

tip = meal_cost * tip_percent / 100
tax = meal_cost * tax_percent / 100
print (round (meal_cost + tip + tax))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.4 Day 3: Intro to Conditional Statements

Objective: Classify the age-independent integer using the challenge's conditions.

```
N = int (input (). strip ())

if N % 2 == 1:
    print("Weird")
elif 2 <= N <= 5:
    print ("Not Weird")
elif 6 <= N <= 20:
    print("Weird")
else:
    print ("Not Weird")
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.5 Day 4: Class vs. Instance

Objective: Create a Person class, validate age, classify it, and show the effect of three years passing.

```
class Person:
    def __init__(self, initialAge):
        if initialAge < 0:
            print("Age is not valid, setting age to 0.")
            self.age = 0
        else:
            self.age = initialAge

    def amIOld(self):
        if self.age < 13:
            print("You are young.")
        elif self.age < 18:
            print("You are a teenager.")
        else:
            print("You are old.")

    def yearPasses(self):
        self.age += 1

t = int(input())
for _ in range(t):
    age = int(input())
    p = Person(age)
    p.amIOld()
    for _ in range(3):
        p.yearPasses()
    p.amIOld()
    print()
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.6 Day 5: Loops

Objective: Print the multiplication table for the given integer.

```
n = int(input())
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.7 Day 6: Let's Review

Objective: Print even-indexed and odd-indexed characters separately for each input string.

```
T = int(input())
for _ in range(T):
    s = input().strip()
    print(s[::2], s[1::2])
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.8 Day 7: Arrays

Objective: Print the array elements in reverse order.

```
n = int(input())
arr = list(map(int, input().split()))
print(*arr[::-1])
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.9 Day 8: Dictionaries and Maps

Objective: Store names and phone numbers, then answer lookups until end-of-file.

```
n = int(input())
phone_book = {}
for _ in range(n):
    name, number = input().split()
    phone_book[name] = number

while True:
    try:
        name = input().strip()
    except EOFError:
        break
    if name in phone_book:
        print(f"{name}={phone_book[name]}")
    else:
        print("Not found")
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.10 Say 'Hello, World!' With Python

Objective: Print the required greeting.

```
if __name__ == '__main__':
    print("Hello, World!")
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.11 Python If-Else

Objective: Apply the HackerRank parity and range conditions.

```
n = int(input().strip())
if n % 2:
    print("Weird")
elif 2 <= n <= 5:
    print("Not Weird")
elif 6 <= n <= 20:
    print("Weird")
else:
    print("Not Weird")
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.12 Arithmetic Operators

Objective: Print the sum, difference, and product of two integers.

```
a = int(input())
b = int(input())
print(a + b)
print(a - b)
print(a * b)
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.13 Division

Objective: Print integer division followed by floating-point division.

```
a = int(input())
b = int(input())
print(a // b)
print(a / b)
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.14 Loops

Objective: Print the square of each integer from zero to n-1.

```
n = int(input())
for i in range(n):
    print(i * i)
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.15 Write a Function

Objective: Return True for leap years and False otherwise.

```
def is_leap(year):
    return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)

year = int(input())
print(is_leap(year))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.16 Print Function

Objective: Print the integers from 1 to n consecutively without separators.

```
n = int(input())
print(*range(1, n + 1), sep='')
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.17 List Comprehensions

Objective: Generate all coordinate triples except those whose sum equals n.

```
x = int(input())
y = int(input())
z = int(input())
n = int(input())

result = [[i, j, k]
          for i in range(x + 1)
          for j in range(y + 1)
          for k in range(z + 1)
          if i + j + k != n]
print(result)
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.18 Find the Runner-Up Score!

Objective: Find the second-highest distinct score.

```
n = int(input())
scores = list(map(int, input().split()))
print(sorted(set(scores))[-2])
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.19 Nested Lists

Objective: Print names with the second-lowest distinct score in alphabetical order.

```
records = []
for _ in range(int(input())):
    name = input()
    score = float(input())
    records.append([name, score])

second_lowest = sorted(set(score for name, score in records))[1]
for name in sorted(name for name, score in records if score == second_lowest):
    print(name)
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.20 Finding the Percentage

Objective: Calculate the queried student's average to two decimal places.

```
n = int(input())
marks_by_student = {}
for _ in range(n):
    row = input().split()
    marks_by_student[row[0]] = list(map(float, row[1:]))

query_name = input().strip()
marks = marks_by_student[query_name]
print(f"{sum(marks) / len(marks):.2f}")
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.21 Lists

Objective: Execute each requested list operation.

```
N = int(input())
items = []
for _ in range(N):
    command, *args = input().split()
    args = list(map(int, args))
    if command == "insert":
        items.insert(args[0], args[1])
    elif command == "print":
        print(items)
    elif command == "remove":
        items.remove(args[0])
    elif command == "append":
        items.append(args[0])
    elif command == "sort":
        items.sort()
    elif command == "pop":
        items.pop()
    elif command == "reverse":
        items.reverse()
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.22 Tuples

Objective: Create a tuple and print its hash.

```
n = int(input())
values = tuple(map(int, input().split()))
print(hash(values))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.23 sWAP cASE

Objective: Swap the case of every alphabetic character.

```
def swap_case(s):
    return s.swapcase()

if __name__ == '__main__':
    print(swap_case(input()))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.24 String Split and Join

Objective: Join whitespace-separated words using hyphens.

```
def split_and_join(line):
    return '-'.join(line.split())

if __name__ == '__main__':
    print(split_and_join(input()))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.25 What's Your Name?

Objective: Print the formatted greeting.

```
def print_full_name(first, last):
    print(f"Hello {first} {last}! You just delved into python.")

if __name__ == '__main__':
    first_name = input()
    last_name = input()
    print_full_name(first_name, last_name)
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.26 Mutations

Objective: Replace the character at the given position.

```
def mutate_string(string, position, character):
    return string[:position] + character + string[position + 1:]

if __name__ == '__main__':
    s = input()
    i, c = input().split()
    print(mutate_string(s, int(i), c))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.27 Find a string

Objective: Count substring matches, including overlapping matches.

```
def count_substring(string, sub_string):
    count = 0
    for i in range(len(string) - len(sub_string) + 1):
        if string[i:i + len(sub_string)] == sub_string:
            count += 1
    return count

if __name__ == '__main__':
    string = input().strip()
    sub_string = input().strip()
    print(count_substring(string, sub_string))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.28 String Validators

Objective: Check for alphanumeric, alphabetic, digit, lowercase, and uppercase characters.

```
s = input()
print(any(c.isalnum() for c in s))
print(any(c.isalpha() for c in s))
print(any(c.isdigit() for c in s))
print(any(c.islower() for c in s))
print(any(c.isupper() for c in s))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.29 Text Alignment

Objective: Print the standard HackerRank H pattern using the supplied odd thickness.

```
thickness = int(input())
c = 'H'

for i in range(thickness):
    print((c * i).rjust(thickness - 1) + c + (c * i).ljust(thickness - 1))
for _ in range(thickness + 1):
    print((c * thickness).center(thickness * 2) +
          (c * thickness).center(thickness * 6))
for _ in range((thickness + 1) // 2):
    print((c * thickness * 5).center(thickness * 6))
for _ in range(thickness + 1):
    print((c * thickness).center(thickness * 2) +
          (c * thickness).center(thickness * 6))
for i in range(thickness):
    print(((c * (thickness - i - 1)).rjust(thickness) + c +
           (c * (thickness - i - 1)).ljust(thickness)).rjust(thickness * 6))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.30 Text Wrap

Objective: Wrap text to the requested width.

```
import textwrap

def wrap(string, max_width):
    return '\n'.join(textwrap.wrap(string, max_width))

if __name__ == '__main__':
    string = input()
    max_width = int(input())
    print(wrap(string, max_width))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.31 Designer Door Mat

Objective: Print the symmetric door mat and welcome line.

```
N, M = map(int, input().split())
for i in range(1, N, 2):
    print(('.|.' * i).center(M, '-'))
print('WELCOME'.center(M, '-'))
for i in range(N - 2, 0, -2):
    print(('.|.' * i).center(M, '-'))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.32 String Formatting

Objective: Print decimal, octal, uppercase hexadecimal, and binary values in aligned columns.

```
def print_formatted(number):
    width = len(format(number, 'b'))
    for i in range(1, number + 1):
        print(str(i).rjust(width),
              format(i, 'o').rjust(width),
              format(i, 'X').rjust(width),
              format(i, 'b').rjust(width))

if __name__ == '__main__':
    print_formatted(int(input()))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.33 Alphabet Rangoli

Objective: Print the alphabet rangoli pattern.

```
import string

def print_rangoli(size):
    alphabet = string.ascii_lowercase
    lines = []
    for i in range(size):
        left = alphabet[size - 1:size - i - 1:-1]
        right = alphabet[size - i - 1:size]
        row = '-'.join(left + right)
        lines.append(row.center(4 * size - 3, '-'))
    print('\n'.join(lines[::-1] + lines[1:]))

if __name__ == '__main__':
    print_rangoli(int(input()))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.34 Capitalize!

Objective: Capitalize each word while preserving spaces in the input.

```
def solve(s):
    return ' '.join(word.capitalize() for word in s.split(' '))

if __name__ == '__main__':
    print(solve(input()))
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.35 The Minion Game

Objective: Count substrings starting at vowels for Kevin and consonants for Stuart.

```
def minion_game(string):
    kevin = 0
    stuart = 0
    n = len(string)
    for i, char in enumerate(string):
        points = n - i
        if char in 'AEIOU':
            kevin += points
        else:
            stuart += points

    if kevin > stuart:
        print("Kevin", kevin)
    elif stuart > kevin:
        print("Stuart", stuart)
    else:
        print("Draw")

if __name__ == '__main__':
    minion_game(input().strip())
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

5.36 Merge the Tools!

Objective: Split the string into chunks and remove repeated characters from each chunk while preserving order.

```
def merge_the_tools(string, k):
    for i in range(0, len(string), k):
        chunk = string[i:i + k]
        seen = set()
        result = []
        for char in chunk:
            if char not in seen:
                seen.add(char)
                result.append(char)
        print(''.join(result))

if __name__ == '__main__':
    string, k = input().strip(), int(input())
    merge_the_tools(string, k)
```

Explanation: The solution applies the core Python concept named in the exercise and follows the expected input/output pattern.

6. Python Concepts Demonstrated

- Variables and built-in data types
- Arithmetic and comparison operators
- if / elif / else control flow
- for loops and iteration
- Functions, return values, classes, and instances
- Lists, tuples, dictionaries, sorting, and sets
- List comprehension and string slicing
- String validation, alignment, and formatting

- Algorithmic problem solving and output verification

7. Coding Best Practices

- Use meaningful variable names and consistent indentation.
- Read the full problem statement before coding.
- Test sample inputs and edge cases.
- Use built-in methods when they make the solution clearer.
- Keep the required function signature if HackerRank provides starter code.
- Check output formatting carefully because extra space or lines can cause failed tests.

8. Conclusion

This project organizes Python practice from basic syntax to data structures, string manipulation, formatting, and medium-level algorithmic problems. The question-by-question solutions provide a reference for learning and revision. The most effective way to use this portfolio is to attempt each exercise independently, compare the logic with the documented solution, and run the code against the platform's test cases.

9. Source Note

Primary source: the supplied Data Python Queries roadmap and the uploaded Python HackerRank project report. The roadmap provides exercise titles, difficulty labels, and subject tags. The solutions in this document are written to correspond to the common HackerRank challenge formats; always follow any additional constraints shown on the live challenge page.